Build a production-grade backend for Amrutam's telemedicine system focusing on scalability, reliability, security, and observability.

Deliverables include code, infra as code, design docs, automated tests, and documentation.

Deliverables
1. Git repo with code and infra
2. README with setup
3. OpenAPI/GraphQL schema
4. Architecture doc (2–4 pages)
5. Tests and CI pipeline
6. Observability setup
7. Security checklist and threat model

Problem Scope

Implement core workflows:
• User lifecycle (auth, roles)
• Doctor availability and booking
• Consultation lifecycle and prescriptions
• Search and filtering • Compliance and audit trails
• Admin analytics. System Requirements
• Scale: 100k daily consultations
• Latency: p95 <200ms reads, <500ms writes
• Availability: 99.95% • Security: encryption, MFA, RBAC
• Observability: metrics, logs, traces
• CI/CD with containerized deployment.

Architecture Tasks

1. High-level architecture and data flow
2. Booking flow sequence diagram
3. ER diagram
4. API schema
5. Retry & backoff strategies
6. Data partitioning
7. Caching and concurrency handling
8. Transaction management and sagas
9. Backup and DR strategy.

Implementation Guidelines
• Language: Node.js, Go, or Python
• DB: PostgreSQL (Redis optional)
• REST or GraphQL API • Modular services with DI
• Idempotency for writes
• Async jobs for heavy tasks
• Rate limiting and input validation
• Secrets via env vars.

Security & Threat Modelling Include OWASP mitigation, attack surface analysis, data classification, encryption, key rotation, audit logs, and dependency scanning.

Data Model Core tables: users, profiles, doctors, availability_slots, consultations, prescriptions, payments, audit_logs.

Evaluation Rubric
Architecture – 20
Core Flows – 20
Code Quality – 15
Security – 10
Observability – 10
Scalability – 10
Infra/CI – 10
Bonus – +10 Fail if critical security or idempotency is missing.

Time Expectation: 4–5days.
Submit repo, docs, and 5-minute demo video.